

Why Localization Matters: Allocation, Not Detection, for Sparse-Contamination Multi-Source RAG Defense

Elias Wang
Independent Researcher
eliaswangac@gmail.com

Abstract

Retrieval-augmented generation (RAG) systems must condition on external retrieved content that may not be trusted. Under *sparse contamination* in multi-source RAG, adversarial content occupies only a small subset of retrieved segments, so effective defense turns on *where* intervention is applied, not only on *what* intervention is applied. The value of localization here is not that it finds attacks. It is that it makes finding them unnecessary. Posed as an allocation decision under a defender-chosen budget rather than as a detection task, the defense needs only a segment ranking that beats chance *within* a single retrieved set, and no calibrated threshold at all. We make this precise with a hit-rate decomposition of expected attack success in which no threshold, calibration, or false-positive rate appears, and then establish it empirically. A localize-then-drop defense built from two deliberately weak, threshold-free signals attains the lowest attack success rate (ASR) of any method tested, 7.5% against 80.4% undefended (2.3% under optimization-based injection, 12.8% under fluent poisoning), at *lower* model-side token cost than applying no defense at all, across five models, two datasets, and two attack families; under knowledge poisoning it *recovers* the answer utility the attack had degraded, inverting the usual robustness-utility trade-off. Holding the intervention fixed and varying only the locator shows that placement, not local modification, carries this benefit. And the signals invert across attack families: each is near-perfect on one and *below* the random-allocation floor on the other, so neither is a usable detector, yet together they allocate effectively. Under sparse contamination, RAG defense should therefore treat localization as an allocation decision rather than a detection task.

CCS Concepts

- **Security and privacy** → **Software and application security**;
- **Computing methodologies** → *Machine learning*.

Keywords

retrieval-augmented generation, indirect prompt injection, RAG poison, sparse contamination, test-time defense, defense allocation

1 Introduction

Large language models (LLMs) are increasingly deployed in retrieval-augmented generation (RAG) systems, where responses are produced by conditioning on externally retrieved documents [8, 16]. By retrieving external context, RAG systems answer questions beyond the model’s parametric memory and ground their responses

in task-relevant evidence. However, this design introduces a fundamental security risk: the model must process external content that may not be trusted [10, 18]. When retrieved documents are controlled, influenced, or poisoned by an adversary, the external context becomes an attack surface rather than only a source of knowledge.

This risk is especially acute under *sparse contamination*. In many RAG settings the retrieved context is assembled from multiple documents or segments, while adversarial content appears in only one or a few of them. Such sparse contamination arises from indirect prompt injection embedded in retrieved documents, or from poisoned evidence inserted into the retrieval corpus [24, 31]. Although these attacks differ in how malicious content enters the pipeline, they share a common structure at defense time: the system receives a mostly benign retrieved context that contains a small number of risky segments.

A test-time defense may inspect, filter, perturb, or rewrite the retrieved context before generation, but stronger defensive processing is not free. It can increase model-side computation, remove useful evidence, distort benign context, and reduce clean-task utility. When only part of the retrieved context is risky, the defender therefore faces an allocation problem: limited defensive effort must be distributed across the retrieved segments, and its placement determines the outcome.

Defense in this setting is usually approached as a *detection* problem: judge each retrieved segment as adversarial or not, then act on whatever is flagged. We argue it is better posed as an *allocation* problem: fix a defensive budget in advance, rank the retrieved segments, and spend that budget on the top of the ranking. The difference is not cosmetic. Detection asks an absolute question and needs a calibrated, corpus-transferable threshold; allocation asks only a relative one within a single retrieved set, with no threshold at all (Figure 1). That is a much weaker requirement, and it is why localization is the right lens for this setting.

Detection versus allocation. Prior localization defenses follow the detection instinct, showing *how* to localize injected content at finer granularity than full-input detection [13, 25]. Our position is more basic: under sparse contamination the hard part is not judging anomaly in the absolute sense but *allocating* a fixed budget to the right region, and this distinction, rather than any particular detector, is what our results turn on. The sharpest evidence is a double dissociation between the two weak signals we use (Table 2): the lexical signal reaches a 98.0% localization hit rate on optimization-based injection but 11.5% on fluent poisoning, below the 18.5% random floor, and the semantic signal is the exact mirror image. Neither is deployable as a detector. Together they still cut ASR from 80.4% to 7.5% across both families.

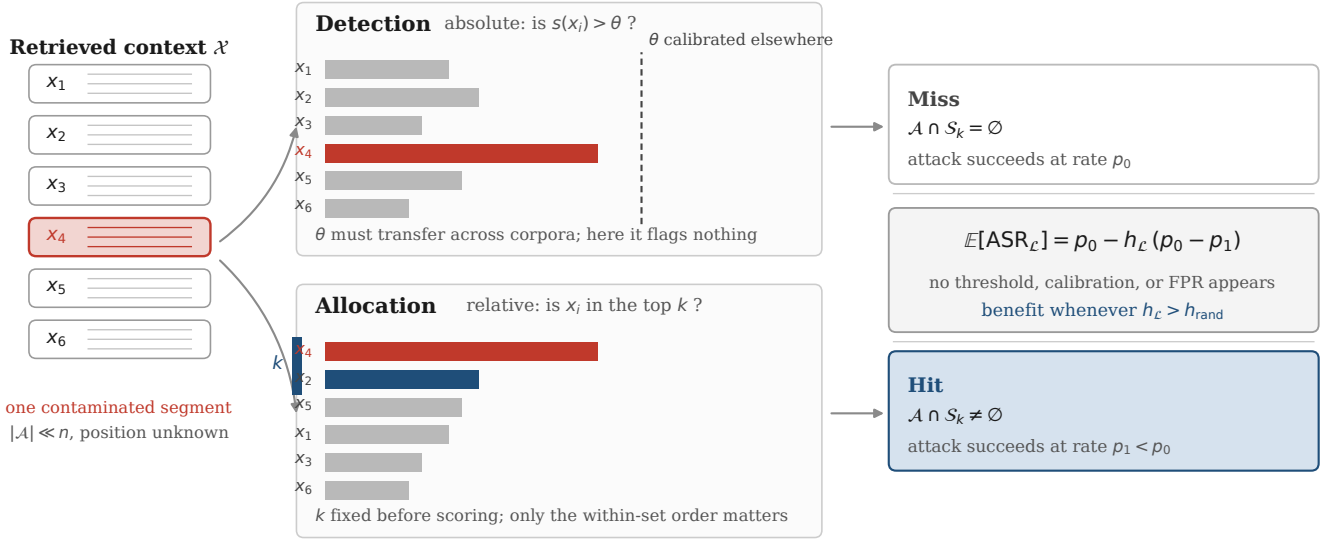


Figure 1: Allocation, not detection. Both rules see the same retrieved set and the same per-segment scores; they differ only in how the selected set is chosen. *Top:* a detection rule compares each score against an absolute threshold θ that must be calibrated on some other distribution and transfer to the content actually retrieved at test time. Here it flags nothing, and a lower θ would flag benign segments instead. *Bottom:* an allocation rule fixes the budget k before any segment is scored and takes the top k of the within-set ranking. The contaminated segment exceeds no threshold, yet it ranks first, so allocation reaches it where detection does not. *Right:* conditioning on whether the selected set reaches the contaminated region yields the hit-rate decomposition of Equation (1), in which no threshold, calibration, or false-positive rate appears.

Contributions. We establish why localization matters through three results, each isolated by a controlled experiment.

- **We frame sparse-contamination RAG defense as an allocation problem** (Section 4) and derive a hit-rate decomposition of expected attack success, $\mathbb{E}[\text{ASR}_{\mathcal{L}}] = p_0 - h_{\mathcal{L}}(p_0 - p_1)$, whose benefit condition depends only on out-ranking a random allocator, with no threshold, calibration, or false-positive control.
- **We show the framework is attack-general even when the signals are not.** The same localize-then-intervene template attains the lowest ASR and the lowest token cost of any method tested, on an optimization-based injection and a fluent poisoning attack, across five models and two datasets (Section 5.2); under poisoning it *recovers* utility rather than trading it away (Section 5.3).
- **We show that placement carries the benefit, and that allocation is easier than detection.** Holding the operator fixed and varying only the locator isolates placement as the source of the effect (Section 5.4), and two threshold-free signals that each fall *below* the random floor on one of the two attack families still allocate effectively (Section 5.5), demonstrating that a good detector is not required.

1.1 Related Work

Detection- and perturbation-based defenses. Prompt injection and jailbreak attacks have motivated test-time defenses that detect, filter, or transform suspicious inputs before generation. One line

treats adversarial prompts as anomalies and applies input-level detection, such as PromptGuard classifiers or perplexity-based scoring [2, 20]. Another perturbs inputs to disrupt optimized adversarial text; SmoothLLM and related baselines exploit the brittleness of GCG-style prompts under small perturbations [12, 22]. A separate family hardens the model or the surrounding system rather than screening inputs: structured-query and preference-optimization training teach the model to ignore instructions embedded in data [3, 4, 21, 23], and system-level designs constrain what retrieved content is permitted to trigger [5]. These input-level and training-level methods reduce the effect of suspicious content, but they act on the full prompt or change what the model will obey; they do not address *where* a test-time defense should concentrate effort across retrieved segments in multi-source RAG.

RAG poisoning defenses. A related line studies defenses against RAG knowledge poisoning and retrieval corruption, a threat surface that spans corpus poisoning of dense retrievers [33] and trigger-based RAG backdoors [27]. RobustRAG uses an isolate-then-aggregate strategy for certifiable robustness when the number of malicious passages is bounded [26]. FilterRAG and ML-FilterRAG target attacks in the PoisonedRAG style by filtering poisoned passages using distributional properties [7]. RAGDefender operates after retrieval with lightweight machine-learning techniques to remove adversarial content without extra LLM inference [15], and TrustRAG clusters retrieved passages to isolate injected content before generation [34]. These defenses act on retrieved passages, but their focus is a specific filtering or aggregation mechanism, not

why sparse contamination creates an allocation problem in the first place.

Localization-based defenses. Most directly related, PromptLocate localizes injected prompts within contaminated inputs, and Web-Sentinel detects and localizes prompt injection in web-agent environments [13, 25]. These works show that suspicious content *can* be localized at finer granularity than full-input detection. We study a different role for localization: not a detection output, but the decision variable that determines where limited defensive effort is spent under sparse contamination.

2 Background and Threat Model

2.1 Indirect Prompt Injection

Indirect prompt injection is a major security threat where untrusted external content steers model behavior after being incorporated into the prompt [10, 18]. The threat is acute for tool-integrated and web agents, where benchmarks show that injected external content can hijack agent actions at scale [6, 30, 32]. In RAG, the threat arises when attackers embed adversarial content into external documents that are later retrieved and inserted into the model context [10]. ObliInjection is an indirect prompt injection attack designed for multi-source RAG settings [24]. Rather than relying on human-readable malicious instructions, it inserts optimized adversarial payloads that can appear in arbitrary segments and positions; through an order-oblivious objective and the orderGCG optimizer, part of the GCG-style discrete-optimization lineage [17, 35], it remains effective when only a small subset of retrieved segments is contaminated [24]. This sparse, segment-level structure makes it a useful case study for localization-aware defense.

2.2 RAG Knowledge Poisoning

Knowledge poisoning targets a different entry point. Instead of placing malicious instructions in already-retrieved documents, the attacker corrupts the external knowledge database before retrieval; once indexed, poisoned texts are retrieved as seemingly relevant evidence and influence generation [36]. PoisonedRAG formulates this as a knowledge-corruption attack that injects a small number of malicious texts to induce an attacker-chosen answer [36]. CorruptRAG studies a more practical single-injection setting, improving stealth while still compromising the answer [31].

Although poisoning and injection differ in how adversarial content enters the pipeline, they create the same test-time challenge: the model receives a collection of retrieved segments in which only a small subset is adversarial or misleading, so the defense problem is not only whether to intervene but where. This shared structure makes poisoning a valuable second attack family: it lets us test whether a localization-aware defense is general or merely tuned to one attack.

2.3 Threat Model and Problem Setting

System setting. We consider a multi-source RAG system in which the model answers a user query q by conditioning on multiple retrieved segments $\mathcal{X} = (x_1, x_2, \dots, x_n)$, where each x_i is a retrieved segment or document chunk. The model input contains q , a system instruction, and the segments in $\mathcal{X}$.

Sparse-contamination setting. Only a small subset of retrieved segments is contaminated. Let $\mathcal{A} \subset \{1, \dots, n\}$ index the contaminated segments, with $|\mathcal{A}| \ll n$. For $i \in \mathcal{A}$, segment x_i may contain injected adversarial content; segments x_j for $j \notin \mathcal{A}$ are benign. The full retrieved context is still passed to the model.

Attack setting. The attacker can influence one or a few segments in $\mathcal{X}$ but not the query, system instruction, model parameters, or full context. The goal is to insert content into the contaminated subset so that the model deviates from the intended task and produces an attacker-desired response.

Defender and objective. The defender observes $\mathcal{X}$ before generation and may apply a test-time defense, but does not know $\mathcal{A}$ in advance. Rather than treating the full prompt as uniformly suspicious, the defense may allocate stronger intervention to selected higher-risk segments. Our objective is not to perfectly identify every contaminated segment, but to examine how localization affects downstream behavior along three axes: attack success rate (ASR) under contaminated retrieval, answer utility (F1), and model-side token cost.

3 The Misalignment of Global Defenses

Before arguing what localization contributes, we make precise why treating the retrieved context as a single global input is the wrong default under sparse contamination. The two standard global strategies, detection and prevention, fail for complementary reasons.

3.1 Global Detection is Diluted by Benign Context

A common intuition treats prompt injection as a distributional anomaly, scored globally by perplexity or a related metric. This is reasonable when malicious content occupies a substantial portion of the input, but unreliable in RAG, where the input is a benign-majority mixture: most segments are normal evidence and only one or a few are contaminated. A single global anomaly score over the concatenated prompt is dominated by the benign majority, so the abnormal signal from a sparse malicious segment is diluted by surrounding normal text.

Making detection finer-grained, by scoring segments independently or with sliding windows, does not resolve the difficulty. It forces a choice of analysis granularity, multiplies test-time cost across many candidate regions, and makes threshold calibration harder, since retrieved content varies across domains, styles, and lengths, so a threshold tuned on one corpus need not transfer. The core problem is a mismatch of granularity: sparse contamination is local, whereas global scoring is dominated by the benign majority.

3.2 Global Prevention is Computationally Wasteful

Preventive defenses such as SmoothLLM exploit the brittleness of adversarially optimized prompts under small perturbations, making smoothing a strong and general primitive. But applying it globally to a RAG prompt is expensive: SmoothLLM requires repeated perturbation and repeated model evaluation, so cost scales with both the number of samples and the total context length. In multi-source RAG, where prompts are long, repeated whole-prompt processing

is costly, and especially wasteful in the benign-majority setting, where most of that cost is spent on regions unlikely to contain the attack.

Taken together, the limitation of global prevention is not merely that it is expensive, but that its strongest operation is applied at a granularity poorly matched to sparse contamination. When the attack is localized, uniform processing treats benign and suspicious regions alike. Localization changes the decision: identify where risk concentrates, then apply stronger intervention only there. This motivates the allocation view we formalize next.

4 Localization as a Defense Allocation Principle

We now make the allocation view precise. Both regimes start from the same input, a query q and a retrieved set $\mathcal{X} = (x_1, \dots, x_n)$, and both end by applying stronger processing to some subset of $\mathcal{X}$. They differ in how that subset is chosen.

A **detection rule** applies a per-segment classifier, $D(x_i) = 1\{s(x_i) > \theta\}$, and processes every segment it flags. The size of the flagged set is determined by the data and the threshold, and may be anywhere from 0 to n . The free parameter is θ , which must be calibrated on some reference distribution and must transfer to the retrieved content actually seen at test time. Correctness is *absolute*: a segment is either anomalous or it is not, independent of what it was retrieved alongside.

An **allocation rule** fixes a budget k in advance, ranks the n segments by a score, and processes the top k . The size of the processed set is a defender choice, not a function of the data. The free parameter is k , not a threshold. Correctness is *relative*: what matters is only whether contaminated segments rank above benign ones *within this retrieved set*.

Two consequences follow, and the rest of this section makes them precise. First, an allocation rule is invariant to any strictly increasing transform of its score, so no calibration is required; only the induced ordering matters. Second, its cost on benign input is bounded by construction at k segments, whereas a detector's clean-side cost is whatever the threshold happens to produce on a distribution it may not have been calibrated for. Section 3 argued that this transfer is exactly what global detection fails to achieve.

4.1 Budgeted Defense Allocation

Let $\mathcal{X} = (x_1, \dots, x_n)$ be the retrieved segments and $\mathcal{A} \subseteq [n]$ the contaminated index set. A test-time defense is an allocation rule deciding which parts of $\mathcal{X}$ receive stronger processing. We represent it with a segment scoring function $\mathcal{L}$ assigning each segment a risk score

$$r_i = \mathcal{L}(q, x_i, \mathcal{X}), \quad i \in [n].$$

As noted above, only the induced ordering matters. Given an intervention budget k , fixed by the defender before any segment is scored, the defense selects

$$\mathcal{S}_k(\mathcal{L}) = \text{TopK}_k(\{r_i\}_{i=1}^n), \quad |\mathcal{S}_k| = k,$$

and applies an intervention operator Φ to the selected region, $\tilde{\mathcal{X}} = \Phi(\mathcal{X}, \mathcal{S}_k)$. The final prompt is built from $\tilde{\mathcal{X}}$ with the original query and system instruction.

This separates three design choices: the scoring rule $\mathcal{L}$ (where effort is allocated), the budget k (how much is processed), and the

operator Φ (what is applied). Full-context defenses are the special case $k = n$: maximal coverage of any contamination, but intervention on *all* benign segments too. Localization-aware defenses use $k < n$ and rely on $\mathcal{L}$ to concentrate effort on higher-risk regions.

4.2 Localization Hit Rate and Attack Mitigation

The key quantity is whether the selected set reaches the contaminated region. Define the hit event and hit rate

$$H_{\mathcal{L}} = 1\{\mathcal{A} \cap \mathcal{S}_k(\mathcal{L}) \neq \emptyset\}, \quad h_{\mathcal{L}} = \Pr(H_{\mathcal{L}} = 1).$$

Let $Y \in \{0, 1\}$ be the attack-success indicator after the defended prompt is evaluated, and write

$$p_0 = \Pr(Y = 1 \mid H_{\mathcal{L}} = 0), \quad p_1 = \Pr(Y = 1 \mid H_{\mathcal{L}} = 1),$$

where reaching an attack-bearing segment does not increase success in expectation, $p_1 \leq p_0$. Both conditionals are defined with respect to the locator under evaluation, since different locators condition on different selected sets; we suppress this dependence in the notation. Conditioning on the hit event,

$$\begin{aligned} \mathbb{E}[\text{ASR}_{\mathcal{L}}] &= (1 - h_{\mathcal{L}})p_0 + h_{\mathcal{L}}p_1 \\ &= p_0 - h_{\mathcal{L}}(p_0 - p_1). \end{aligned} \quad (1)$$

Equation (1) is elementary, and deliberately so. Its content is not mathematical depth but a change of objective: it shows that the quantity governing mitigation is a within-set *ranking* statistic, and that the calibration machinery detection-based defenses are built around does not enter the outcome at all. A defense can therefore be effective while being, by any standard classification measure, a poor detector. The term $h_{\mathcal{L}}$ measures how often the defense reaches the contaminated region; $p_0 - p_1$ measures how much the intervention helps once it does. Under a fixed Φ and budget, improving localization reduces expected ASR whenever reaching the contaminated segment makes the intervention effective ($p_1 < p_0$).

Why this means allocation is easier than detection. This is where the definitional difference above becomes quantitative. No threshold, calibration, or false-positive rate appears in Equation (1). A budget-matched random allocator R that selects k indices uniformly at random has hit rate

$$h_{\text{rand}} = 1 - \frac{\binom{n-a}{k}}{\binom{n}{k}}, \quad a = |\mathcal{A}|,$$

which reduces to $h_{\text{rand}} = k/n$ when $|\mathcal{A}| = 1$. Substituting,

$$\mathbb{E}[\text{ASR}_{\text{rand}}] - \mathbb{E}[\text{ASR}_{\mathcal{L}}] = (h_{\mathcal{L}} - h_{\text{rand}})(p_0 - p_1).$$

Allocation therefore improves over random *whenever the localizer out-ranks a random allocator*, $h_{\mathcal{L}} > h_{\text{rand}}$, and the intervention has nonzero effect, $p_0 > p_1$. A localizer does not need to be a good detector, or even a good classifier in the absolute sense; it only needs a segment ordering that beats chance *within the retrieved set*. This is a far lower bar than the transferable-threshold requirement that global detection fails (Section 3), and it is the reason localization is the right frame. It also names the failure mode: as $h_{\mathcal{L}} \rightarrow h_{\text{rand}}$, or as $p_0 - p_1 \rightarrow 0$, the benefit vanishes.

This decomposition directly drives our experiments. Section 5.2 evaluates the template end to end against global baselines on both

attack families; fixing Φ and varying the locator isolates $h_{\mathcal{L}}$ (Section 5.4); measuring each signal's standalone hit rate against the random floor isolates the detection-versus-allocation gap (Section 5.5); and fixing the locator while varying Φ isolates $p_0 - p_1$ (Section 5.6).

5 Experiments

5.1 Setup

The localizer is deliberately weak. Because our claim is about allocation rather than detection, we instantiate the localizer $\mathcal{L}$ with the cheapest signals we can, precisely to test how weak a localizer can be while allocation still helps. We use two threshold-free, per-segment signals, neither of which performs an additional model forward pass at defense time (Algorithm 1):

- **Lexical signal** r^{freq} : low-frequency token count of each segment. Let $\mathcal{T}$ be the target model's tokenizer, τ the percentile threshold below which a token is low-frequency, and $\mathcal{T}(x_i) = (t_{i,1}, \dots, t_{i,m_i})$; then $r_i^{\text{freq}} = \sum_{j=1}^{m_i} \mathbf{1}[\text{freq}(t_{i,j}) < \tau]$. The signal is motivated by an empirical artifact of GCG-style optimization [35], in which optimized payloads drift toward low-frequency tokens. An injected payload contributes a roughly fixed number of low-frequency tokens irrespective of segment length, so an absolute count preserves that margin where a length-normalised density would divide it away.
- **Semantic signal** r^{cos} : absolute deviation of each segment's query-relevance from the mean relevance of the retrieved set. Let $E(\cdot)$ be the retriever's encoder; with $d_i = 1 - \cos(E(x_i), E(q))$ and $\bar{d} = \frac{1}{n} \sum_i d_i$, $r_i^{\text{cos}} = |d_i - \bar{d}|$. The deviation is two-sided: a contaminated segment may be unusually unrelated to the query (off-topic injection) or unusually related (a poisoned passage optimized to win retrieval). Because E is the retriever's own encoder, the embeddings are already available, so this adds no forward pass at defense time.

Each signal is min-max normalized across the n segments, $\text{MINMAX}(v)_i = (v_i - \min_j v_j) / (\max_j v_j - \min_j v_j + \epsilon)$, and the two normalized scores are combined by an element-wise maximum, $r_i = \max(r_i^{\text{freq}}, r_i^{\text{cos}})$, so a segment is selected when *either* signal flags it as anomalous. This union behavior is what the two-signal design needs: the signals fail on different attack families (Section 5.5), so a segment anomalous to only one of them should still be selected, and no weighting parameter is introduced. The localizer then selects the top $k = \lceil \rho n \rceil$ segments (default $\rho = 0.2$); only these segments are modified. We stress that neither signal is a detector; Section 5.5 shows each is *below* the random floor on one attack family. The intervention operator Φ is localized segment dropping unless stated otherwise: selected segments are removed and the rest are kept in order, with no auxiliary model calls, so any effect is attributable to *where* intervention is applied rather than to added model-based processing.

Target LLMs. Qwen3-4B, Gemma-2-2B, Llama3-8B, Mistral-7B, and Falcon3-7B [1, 9, 14, 19, 28].

Datasets and samples. Two multi-hop QA datasets, HotpotQA [29] and 2WikiMultiHopQA [11], provide benign multi-source contexts. For each dataset-model pair we use 200 attacked examples for ASR and the corresponding benign examples for clean-task F1, with

Algorithm 1 Multi-Signal Segment Localizer (MULTISIGNALIZE)

Require: Query q ; segments $\mathcal{X} = (x_1, \dots, x_n)$; tokenizer $\mathcal{T}$; frequency table $\text{freq}(\cdot)$; threshold τ ; encoder $E(\cdot)$; fraction ρ

Ensure: Selected index set $\mathcal{S}_k$

```

1:  $k \leftarrow \lceil \rho n \rceil$ 
   // lexical signal
2: for  $i = 1$  to  $n$  do
3:    $(t_{i,1}, \dots, t_{i,m_i}) \leftarrow \mathcal{T}(x_i)$ 
4:    $r_i^{\text{freq}} \leftarrow \sum_j \mathbf{1}[\text{freq}(t_{i,j}) < \tau]$ 
5: end for
   // semantic signal (reuse retriever embeddings)
6:  $e_q \leftarrow E(q)$ 
7: for  $i = 1$  to  $n$  do
8:    $d_i \leftarrow 1 - \cos(E(x_i), e_q)$ 
9: end for
10:  $\bar{d} \leftarrow \frac{1}{n} \sum_i d_i$ ;  $r_i^{\text{cos}} \leftarrow |d_i - \bar{d}|$  for all  $i$ 
   // scale-free fusion
11:  $\tilde{r}^{\text{freq}} \leftarrow \text{MINMAX}(r^{\text{freq}})$ ;  $\tilde{r}^{\text{cos}} \leftarrow \text{MINMAX}(r^{\text{cos}})$ 
12: for  $i = 1$  to  $n$  do
13:    $r_i \leftarrow \max(\tilde{r}_i^{\text{freq}}, \tilde{r}_i^{\text{cos}})$ 
14: end for
15:  $\mathcal{S}_k \leftarrow \text{TopK}_k(\{r_i\}_{i=1}^n)$ 
16: return  $\mathcal{S}_k$ 

```

exactly one contaminated segment per attacked sample. All defenses see the same queries, contexts, contaminated positions, and payloads (Appendix A).

Attack families. We evaluate two sparse-contamination attacks: **ObliInjection**, an optimization-based indirect prompt injection [24], and **CorruptRAG**, a fluent single-injection knowledge-poisoning attack [31]. For ObliInjection we place an optimized payload in one segment behind a benign wrapper; for CorruptRAG we insert a single fluent poisoned passage. Exactly one segment is contaminated per attacked sample.

Compared methods. Global baselines are Full-Prompt Perplexity (Full-PPL) [2], Windowed Perplexity (Win-PPL) [12], and full-context SmoothLLM [22]; Segment-Level Perplexity (Seg-PPL) is a segment-granularity filtering baseline. Our localize-then-intervene defense is **SL-Drop** (Sparse Localization then Drop), which uses the two-signal localizer to select high-risk segments and drops them. Perplexity thresholds are calibrated to a 5% clean false-positive rate; SmoothLLM uses 10 character-swap perturbations aggregated by majority vote.

Metrics. **ASR**: fraction of attacked samples whose response satisfies the attacker target (lower is better). **Clean F1**: answer F1 against ground truth on the benign counterparts of each attack family's evaluation samples (higher is better). **Under-attack F1**: answer F1 on attacked samples; we report it for CorruptRAG, whose harm is a corrupted answer rather than a suppressed one, so utility under attack is the informative axis there. In per-model tables, No-defense rows and the ObliInjection blocks therefore report clean F1 (the utility ceiling), while defended methods under CorruptRAG report under-attack F1, i.e. how much of that ceiling the defense recovers.

Table 1: The same template wins on both attacks at lowest cost. ASR and token cost are pooled over both attack families, all models, and both datasets; clean F1 is measured under benign retrieval on the ObliInjection evaluation sets. Lower ASR and token cost are better; higher clean F1 is better.

Method	Avg. ASR (%)	Avg. clean F1	Avg. token cost	Rel. cost
No def.	80.40	0.3696	1208.40	1.00×
Full-PPL	73.70	0.3688	2928.30	2.42×
Win-PPL	71.90	0.3113	5251.00	4.35×
SmoothLLM	39.40	0.1851	16831.30	13.93×
Seg-PPL	76.53	0.3559	2987.10	2.47×
SL-Drop	7.50	0.3287	903.75	0.75×

Model-side token cost: total input tokens processed by model-based defense steps and generation (lower is better).

5.2 Main Result: One Template, Two Attack Families, Lowest Cost

We begin with the end-to-end comparison, so that the mechanism analyses that follow are explaining a result the reader has already seen. The question here is whether a localize-then-intervene template built from signals that are individually poor can nonetheless beat strong global baselines, and whether it does so on attacks that break *different* signals.

Table 1 reports averages across models, datasets, and both attack families. Global perplexity defenses provide limited mitigation under sparse contamination and increase cost through added scoring; full-context SmoothLLM mitigates more strongly but at the largest utility cost (0.185 clean F1) and the largest token cost (13.93×). SL-Drop, using only the two weak signals for allocation, achieves the lowest ASR (7.50% pooled against 80.40% undefended; 2.25% on ObliInjection and 12.75% on CorruptRAG) at the lowest token cost (0.75×), cheaper than applying no defense, because it removes segments before generation rather than adding model-based processing. The per-model, per-attack breakdown (Table 5) confirms the pattern: SL-Drop attains the lowest ASR in 19 of 20 model-dataset-attack cells (the exception is Falcon3-7B on HotpotQA under ObliInjection, where full-context SmoothLLM reaches 1.0% at roughly 14× the cost), including on CorruptRAG, where the lexical signal is below random but the semantic signal carries the allocation.

The generality claim is therefore established at the level of the *framework*, not the localizer. The two signals behind SL-Drop have complementary blind spots, quantified in Section 5.5, yet the template built on them wins on both families. The remaining subsections explain why.

5.3 Localization Recovers Utility Under Poisoning

A striking effect appears on the poisoning family. On CorruptRAG the poisoned passage actively corrupts the answer, so the informative utility axis is under-attack F1 (Section 5.1), and dropping the poisoned passage does not merely avoid degrading utility; it *recovers* it. Without defense, under-attack F1 collapses far below the clean ceiling: on HotpotQA the mean falls from 0.410 (clean) to

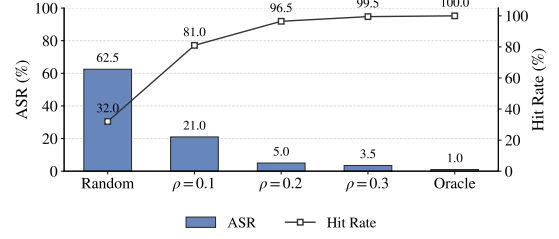


Figure 2: Placement carries the benefit. Mistral-7B, lexical signal, drop operator. Varying only the locator with the intervention fixed. Bars show ASR, the line shows localization hit rate. Random allocation is budget-matched to the locator’s selection and leaves ASR high despite the same budget; as hit rate rises with the targeted locator, ASR falls sharply; oracle localization bounds the effect. Lower ASR and higher hit rate are better.

0.180 (attacked). SL-Drop lifts it back to 0.368, recovering 82% of the utility the attack had destroyed, and per model the recovery can be nearly complete (Qwen3-4B 0.212 $\rightarrow$ 0.364 against a 0.512 ceiling; Llama3-8B 0.229 $\rightarrow$ 0.382 against a 0.387 ceiling; Table 4, drop column, and Table 5). Full-context SmoothLLM instead degrades under-attack F1 below even the undefended level (mean 0.153). This inverts the usual trade-off between robustness and utility, and it follows directly from allocation: by confining intervention to the contaminated region, the defense removes the harmful segment without disturbing the benign evidence the model needs. A global defense cannot obtain this effect, because whatever it does to the poisoned passage it also does to the evidence the answer depends on.

5.4 Placement, Not Local Modification, Carries the Benefit

Having established that the template works, we now isolate *why*. Equation (1) predicts that mitigation is governed by the hit rate h_L , the rate at which the intervention reaches the contaminated segment. This yields a testable consequence: the benefit of a localized defense should come from *where* the intervention lands, not from modifying a fixed-size subset of the context. If placement were irrelevant, a budget-matched random allocation would mitigate the attack as well as a targeted one. We test this by holding the intervention operator fixed and varying only the locator. Concretely, we compare the lexical (frequency) signal at $\rho \in \{0.1, 0.2, 0.3\}$ against a budget-matched *random* allocator and an *oracle* that always selects the contaminated segment. The random allocator controls for the selection budget, and the oracle upper-bounds the mitigation available to this intervention mechanism. We report localization hit rate h_L and the resulting ASR.

Figure 2 shows a direct, monotone relationship between hit rate and mitigation, matching the prediction of Equation (1). Budget-matched random allocation attains a low hit rate and leaves ASR high, so modifying a fixed-size region is not sufficient on its own. As the locator’s hit rate rises, ASR falls sharply, and oracle localization, which always reaches the contaminated segment, bounds the

Table 2: Single signals are not detectors, yet each beats random on one family. Standalone localization hit rate (%) on Mistral-7B, HotpotQA, $\rho = 0.2$. Each signal is near-perfect on one attack and *below the random floor* on the other; neither is safe to deploy alone. The random floor is empirical: with $k = \lceil \rho n \rceil$ and n varying across samples, it differs across attack families.

Signal	OblInjection	CorruptRAG
r^{freq} (lexical)	98.0	11.5
r^{cos} (semantic)	10.5	99.0
Random floor	21.5	18.5

achievable mitigation. Because the intervention operator is held fixed, the only variable that changed is *where* it was applied, which isolates placement, rather than local modification per se, as the source of the benefit. The allocation term $h_{\mathcal{L}}$ in Equation (1) is therefore doing the work.

5.5 Allocation Requires Only Ranking, Not Detection

The previous subsection establishes that reaching the contaminated segment is what matters. A natural next assumption is that reaching it requires a good detector. Equation (1) contradicts this: allocation needs only $h_{\mathcal{L}} > h_{\text{rand}}$, a ranking that beats chance within the retrieved set, not a calibrated detector. This subsection shows that the signals behind the Section 5.2 result are, individually, far too weak to serve as detectors.

To separate detection from ranking, we measure the standalone localization hit rate of each weak signal on both attack families, budget-matched at $\rho = 0.2$, against the random floor. A genuine detector would generalize across attacks, whereas a signal that only provides useful *rank* on the family it was designed for need not.

Result: the signals invert. Table 2 shows a clean double dissociation. The lexical signal is near-perfect on optimization-based OblInjection (98.0%) but falls *below* the random floor on fluent CorruptRAG poisoning (11.5% vs. 18.5%). The semantic signal is the mirror image: near-random on OblInjection (10.5% vs. 21.5%) and near-perfect on CorruptRAG (99.0%). The reason is mechanistic: optimized payloads contain rare tokens that the lexical signal catches but that make fluent poisoning invisible to it, while a fabricated poisoned passage sits at anomalous query relevance that the semantic signal catches but that optimized-yet-on-topic injection does not trigger. Neither signal is a detector; each is *anti-correlated* with contamination on the family it was not designed for.

Yet allocation still works. This is the pair of signals that produced the Table 1 result: 7.5% ASR at sub-1 $\times$ cost. Effective allocation therefore does not require a good detector; it requires only a segment ranking that beats random *within* the retrieved set, exactly as Equation (1) says. The weakness of the signals is not a limitation of the study; it is the evidence for the claim. It also carries a design implication: because no single signal is safe when the defender cannot know the attack in advance, a localizer for open settings

Table 3: Suppressing a signal lowers hit rate, and costs the attacker. Frequency-adaptive OblInjection payloads on Mistral-7B, with localized drop under the lexical-only locator at $\rho = 0.2$ (the Figure 2 configuration). Lower hit rate raises defended ASR as Equation (1) predicts, but undefended ASR also falls.

Payload	Avg. Risky Tokens	Hit Rate (%)	ASR w/o Def. (%)	ASR w/ Lex-Drop (%)
Original OblInj.	6	96.5	86.50	0.00
Freq.-adaptive	1	27.5	37.00	28.00

should combine signals that fail *independently*, so that an attacker suppressing one is still exposed by another.

Adaptive pressure. To probe the boundary of this claim we stress the lexical signal directly. Table 3 constructs frequency-adaptive OblInjection payloads that avoid low-frequency tokens, and evaluates localized drop with the lexical-only locator, since the lexical signal is the one under attack. As expected from Equation (1), suppressing the signal lowers the hit rate (96.5% $\rightarrow$ 27.5%) and the mitigation weakens accordingly. Two things are worth noting. First, the degradation is exactly the predicted mechanism (lower $h_{\mathcal{L}}$, higher ASR), not a breakdown of the framework. Second, suppressing the signal also *costs the attacker*: undefended ASR drops from 86.5% to 37.0%, because avoiding low-frequency tokens moves the optimizer away from a more effective attack region. Evading a localization signal is not free for the adversary. This motivates combining independent signals, so that avoiding one does not avoid all.

5.6 Allocation and the Operator are Distinct Factors

Localization decides *where*; the operator decides *how strongly*. Equation (1) keeps these separate as $h_{\mathcal{L}}$ and $p_0 - p_1$, and the final experiment confirms the separation is real rather than notational. We fix the two-signal locator at $\rho = 0.2$ and vary only Φ across three operators: *drop* (remove the selected segment), *swap* (retain it but randomly permute its words), and *smooth* (apply the SmoothLLM primitive to the selected segments only). Because the locator is identical, the hit rate is identical cell by cell, so the comparison isolates $p_0 - p_1$ exactly.

Table 4 reports all three operators on both attack families. Drop attains the lowest ASR on both, and the gap widens sharply on poisoning: mean ASR is 2.70% (drop), 2.90% (swap), and 6.10% (smooth) on OblInjection, but 13.40%, 17.70%, and 49.70% on CorruptRAG. Cost orders the same way on both families, with drop cheapest, swap modestly more expensive because the selected segment is retained in context, and smooth an order of magnitude above either.

Perturbation is attack-dependent; removal is not. The smoothing column is the informative one. Character-level perturbation works by breaking the brittleness of an optimized payload, so it is effective only when the contaminated content is optimized. On OblInjection it recovers most of the available mitigation; on fluent CorruptRAG poisoning it leaves ASR near 50% despite reaching the contaminated

segment in 87.2% of cases. The poisoned passage remains semantically intact after perturbation, so $p_0 - p_1$ collapses even though $h_{\mathcal{L}}$ is unchanged. Removal has no such dependence: it deletes the content whatever form it takes, so $p_0 - p_1$ stays large across both families. Operator strength in the sense of computational effort is therefore not the same as operator effectiveness. Under accurate localization the cheapest operator is also the most effective one.

Operator choice still trades ASR against utility. The one exception is answer F1 under poisoning, where swap attains 0.4092 against drop's 0.3684 while conceding 4.3 ASR points. Retaining a permuted segment preserves some of its lexical content for the model to condition on, whereas removal discards it entirely. Placement is fixed in both cases, so this is purely an operator effect, and it shows that fixing $h_{\mathcal{L}}$ does not fix the robustness-utility profile. Allocation and intervention strength are therefore distinct design factors, both present in Equation (1), and a defense should be reported as a choice of each.

Table 4: Allocation and operator are distinct factors. Identical two-signal locator ($\rho = 0.2$) on HotpotQA; only Φ varies, so the hit rate is identical across operators within each row. F1 reports clean-task F1 for ObliInjection and under-attack F1 for CorruptRAG (Section 5.1). Drop attains the lowest ASR on both attack families at the lowest cost; smoothing degrades sharply on fluent poisoning despite an unchanged hit rate. Best value per metric block in bold.

Attack	Model	Hit (%)	ASR (%)			Answer F1			Token cost		
			Drop	Swap	Smooth	Drop	Swap	Smooth	Drop	Swap	Smooth
OblInjection	Qwen3-4B	98.5	1.00	0.50	3.50	0.4504	0.4984	0.3829	1171	1596	17644
	Gemma-2-2B	85.0	5.00	6.50	6.00	0.3583	0.3045	0.3483	1132	1566	17329
	Llama3-8B	100.0	0.00	0.00	9.50	0.3989	0.3917	0.3505	1113	1518	16858
	Mistral-7B	99.0	1.00	1.00	3.00	0.2750	0.2158	0.1837	1277	1738	19233
	Falcon3-7B	92.5	6.50	6.50	8.50	0.3816	0.3688	0.3570	1183	1604	17795
	Mean	95.0	2.70	2.90	6.10	0.3728	0.3558	0.3245	1175	1604	17772
CorruptRAG	Qwen3-4B	81.5	17.00	18.50	46.00	0.3641	0.4653	0.2774	1058	1336	14443
	Gemma-2-2B	92.0	13.00	15.50	61.50	0.4189	0.4371	0.2284	998	1302	14207
	Llama3-8B	82.0	14.00	18.00	49.50	0.3816	0.4306	0.2856	1009	1278	13861
	Mistral-7B	92.5	12.00	18.50	49.00	0.2632	0.3108	0.2015	1183	1460	15641
	Falcon3-7B	88.0	11.00	18.00	42.50	0.4141	0.4022	0.3305	1059	1345	14618
	Mean	87.2	13.40	17.70	49.70	0.3684	0.4092	0.2647	1062	1344	14554

6 Discussion

These results converge on a single explanation for why localization matters under sparse contamination: it converts a hard problem into an easy one. Detection requires deciding whether a segment is anomalous in an absolute, corpus-transferable sense, the judgment global defenses fail to make reliably (Section 3). Allocation requires only deciding whether a segment is more suspect than its neighbors, a judgment Equation (1) shows is sufficient and that Table 2 shows even signals that fall below random on other attacks can supply. Three consequences follow. Placement is the operative variable (Section 5.4): the same intervention becomes effective when routed to the contaminated segment and useless when scattered at random. Signal quality is not the bottleneck (Section 5.5): two signals that each fail on half the attacks still allocate effectively, because they need only beat chance where they work; the honest caveat is that a signal driven to chance by an adaptive attacker loses its benefit, though suppressing it also costs the attacker (Table 3). The framework generalizes where the signals do not (Section 5.2): the same template wins on two attack families at the lowest cost of any method tested, and under poisoning it recovers utility rather than trading it away (Section 5.3).

The broader implication is methodological. A defense for sparse-contamination RAG should be evaluated as two separable components, the allocation rule that decides *where* to act and the operator that decides *how*, rather than as a single monolithic detector. The localizer in this paper is deliberately minimal so that the allocation effect is unambiguous; stronger localizers and operators are complementary, not competing, directions.

7 Conclusion and Future Work

We studied test-time defense in sparse-contamination multi-source RAG and argued that the right frame is *allocation*, not detection. When adversarial content is confined to a few retrieved segments, *where* to intervene matters as much as *what* intervention to apply, and this placement decision can be driven by signals far too weak to serve as detectors. A hit-rate decomposition shows the benefit depends only on out-ranking a random allocator; two threshold-free signals, each below random on one of two attack families, cut

ASR from 80.4% to 7.5% across both families at lower cost than no defense, while recovering utility under poisoning; and a locator-quality sweep confirms that placement, not local modification, carries the effect.

Future work. Three directions follow naturally. First, stronger and more independent localization signals (semantic inconsistency, instruction-likeness, retrieval provenance, or learned segment-level scoring) and their behavior under adaptive attack, since Equation (1) makes the benefit contingent on h_L . Second, stronger intervention operators (rewriting, paraphrasing, localized isolate-then-aggregate) that make $p_0 - p_1$ large while remaining affordable precisely *because* they are localized. Third, generalization beyond exactly one contaminated segment and beyond QA-style retrieval, to multi-segment contamination, other injection forms, and tool-using agents. More broadly, our results suggest RAG security should treat localization as a first-class allocation problem, deciding where to spend defensive effort before applying costly or disruptive processing.

A Experimental Configuration

Table 6: Experimental configuration.

Item	Setting
Datasets	HotpotQA, 2WikiMultihopQA
Samples per dataset	$N = 200$
Contaminated segments / example	1
Contamination ratio	$1/n$
Contaminated segment selection	Random
Attack families	OblInjection (orderGCG); CorruptRAG
OblInjection target	Only Print: The answer is unknown.
CorruptRAG target	Attacker-chosen incorrect answer
Clean utility metric	Answer F1 on no-attack examples
Generation max tokens	20
Temperature	0.1

Table 5: Full per-model results. Cells behind the averages in Table 1. Each model spans three columns: attack success rate (ASR, %), answer F1, and model-side token cost (total input tokens). F1 in the ObliInjection blocks and in every No-defense row is clean-task F1, the utility ceiling; F1 for defended methods in the CorruptRAG blocks is under-attack F1, showing how much of that ceiling each defense recovers (Section 5.1). Clean F1 is computed on each attack family’s own benign counterpart set. Lower ASR and cost are better; higher F1 is better. Bold marks the best value per column block among defenses; the No-defense row is the undefended reference.

Attack	Dataset	Method	Qwen3-4B			Gemma-2-2B			Llama3-8B			Mistral-7B			Falcon3-7B		
			ASR	F1	Cost	ASR	F1	Cost	ASR	F1	Cost	ASR	F1	Cost	ASR	F1	Cost
OblInjection	HotpotQA	No def.	94.0	0.533	1581	50.0	0.499	1551	100.0	0.192	1503	86.5	0.235	1723	85.5	0.583	1590
		Full-PPL	92.0	0.538	4482	48.5	0.477	4402	92.5	0.191	4185	77.0	0.235	4893	77.0	0.585	4516
		Seg-PPL	93.0	0.477	4474	49.5	0.426	4371	96.0	0.185	4460	86.5	0.234	4893	63.5	0.594	4637
		Win-PPL	84.5	0.531	8562	45.0	0.500	8450	80.0	0.182	6739	64.0	0.240	9336	66.5	0.583	8654
		SmoothLLM	20.5	0.287	22202	7.0	0.187	21497	10.0	0.118	21486	2.5	0.125	23959	1.0	0.292	22407
		SL-Drop	1.0	0.450	1171	5.0	0.358	1132	0.0	0.399	1113	1.0	0.275	1277	6.5	0.382	1183
	2WikiMulti.	No def.	100.0	0.331	1023	87.0	0.510	1000	98.5	0.068	966	98.0	0.149	1102	89.5	0.596	1013
		Full-PPL	90.0	0.336	2868	77.0	0.514	2807	79.5	0.069	2717	68.5	0.147	3101	76.0	0.597	2843
		Seg-PPL	100.0	0.356	2730	87.0	0.468	2593	91.0	0.075	2776	98.5	0.150	2967	47.0	0.595	2926
		Win-PPL	92.5	0.186	4995	82.0	0.208	4867	86.0	0.064	5000	86.5	0.104	5563	61.5	0.515	5094
		SmoothLLM	61.5	0.165	14133	25.5	0.185	13700	33.5	0.080	13551	25.5	0.086	15238	12.0	0.326	14207
		SL-Drop	0.0	0.247	659	3.5	0.446	605	0.0	0.060	620	1.5	0.135	759	4.0	0.535	618
CorruptRAG	HotpotQA	No def.	64.5	0.512	1334	88.5	0.404	1302	59.5	0.387	1276	59.5	0.319	1460	61.5	0.428	1345
		Full-PPL	62.0	0.218	2611	84.5	0.089	2547	57.0	0.235	2499	56.5	0.120	2853	59.5	0.245	2634
		Seg-PPL	64.5	0.209	2745	88.0	0.087	2692	60.5	0.229	2657	59.5	0.121	2949	60.0	0.263	2784
		Win-PPL	61.5	0.202	4549	84.5	0.083	4445	58.0	0.215	4372	57.0	0.110	4946	58.5	0.241	4591
		SmoothLLM	44.0	0.196	18622	60.5	0.147	17972	44.0	0.201	18020	48.5	0.088	20089	54.0	0.131	18824
		SL-Drop	17.0	0.364	1058	13.0	0.419	998	14.0	0.382	1009	12.0	0.263	1183	11.0	0.414	1059
	2WikiMulti.	No def.	73.0	0.439	878	86.5	0.425	859	86.5	0.449	832	71.0	0.125	953	68.5	0.524	877
		Full-PPL	71.5	0.218	1719	84.5	0.069	1683	82.0	0.082	1625	69.5	0.067	1861	69.0	0.255	1720
		Seg-PPL	73.5	0.198	1814	87.5	0.054	1785	87.0	0.062	1750	69.0	0.073	1912	69.0	0.248	1827
		Win-PPL	69.0	0.198	2964	82.5	0.059	2911	83.0	0.067	2834	69.0	0.068	3183	66.5	0.245	2965
		SmoothLLM	67.5	0.072	12106	84.0	0.120	11722	69.5	0.117	11637	63.5	0.036	13041	53.5	0.277	12213
		SL-Drop	10.0	0.407	725	8.5	0.339	710	14.0	0.410	686	10.5	0.108	785	17.5	0.466	725

B Full Per-Model Results

Table 5 reports the complete per-model results behind the averages in Table 1.

References

- [1] E. Almazrouei, H. Alobeidli, A. Alshamsi, et al. 2023. The Falcon Series of Open Language Models. *arXiv preprint arXiv:2311.16867* (2023).
- [2] G. Alon and M. J. Kamfonas. 2023. Detecting Language Model Attacks with Perplexity. *arXiv preprint arXiv:2308.14132* (2023).
- [3] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: Defending Against Prompt Injection with Structured Queries. In *USENIX Security Symposium*.
- [4] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*.
- [5] Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025).
- [6] Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [7] K. Edemacu, V. M. Shashidhar, M. Tuape, D. Abudu, B. Jang, and J. W. Kim. 2025. Defending Against Knowledge Poisoning Attacks During Retrieval-Augmented Generation. *arXiv preprint arXiv:2508.02835* (2025).
- [8] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, 6491–6501.
- [9] Gemma Team. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118* (2024).
- [10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, 79–90.
- [11] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa. 2020. Constructing a Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*.
- [12] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein. 2023. Baseline Defenses for Adversarial Attacks Against Aligned Language Models. *arXiv preprint arXiv:2309.00614* (2023).
- [13] Y. Jia, Y. Liu, Z. Shao, J. Jia, and N. Z. Gong. 2025. PromptLocate: Localizing Prompt Injection Attacks. *arXiv preprint arXiv:2510.12252* (2025).
- [14] A. Q. Jiang, A. Sablayrolles, A. Mensch, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [15] M. Kim, H. Lee, and H. Koo. 2025. Rescuing the Unpoisoned: Efficient Defense Against Knowledge Corruption Attacks on RAG Systems. In *Annual Computer Security Applications Conference (ACSAC)*.
- [16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.
- [17] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [18] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 1831–1847.
- [19] Llama Team, AI @ Meta. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [20] Meta AI. 2025. Llama Prompt Guard 2. Model card. <https://www.llama.com/docs/model-cards-and-prompt-formats/prompt-guard/>
- [21] Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. 2024. Jatmo: Prompt Injection Defense by Task-Specific Finetuning. In *European Symposium on Research in Computer Security (ESORICS)*.
- [22] A. Robey, E. Wong, H. Hassani, and G. J. Pappas. 2023. SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks. *arXiv preprint arXiv:2310.03684* (2023).
- [23] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv preprint arXiv:2404.13208* (2024).
- [24] R. Wang, Y. Jia, and N. Z. Gong. 2025. OblInjection: Order-Oblivious Prompt Injection Attack to LLM Agents with Multi-Source Data. *arXiv preprint*

- arXiv:2512.09321* (2025).
- [25] X. Wang, Y. Liu, Z. Wang, D. Song, and N. Z. Gong. 2026. WebSentinel: Detecting and Localizing Prompt Injection Attacks for Web Agents. *arXiv preprint arXiv:2602.03792* (2026).
 - [26] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal. 2024. Certifiably Robust RAG Against Retrieval Corruption. *arXiv preprint arXiv:2405.15556* (2024).
 - [27] Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. 2024. BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models. *arXiv preprint arXiv:2406.00083* (2024).
 - [28] A. Yang, A. Li, B. Yang, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).
 - [29] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2369–2380.
 - [30] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics (ACL)*.
 - [31] B. Zhang, Y. Chen, Z. Liu, L. Nie, T. Li, Z. Liu, and M. Fang. 2026. Practical Poisoning Attacks Against Retrieval-Augmented Generation. In *ACM Symposium on Access Control Models and Technologies (SACMAT)*. To appear.
 - [32] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents. In *International Conference on Learning Representations (ICLR)*.
 - [33] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen. 2023. Poisoning Retrieval Corpora by Injecting Adversarial Passages. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
 - [34] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, Zhaoyang Wang, Hamed Haddadi, and Emine Yilmaz. 2025. TrustRAG: Enhancing Robustness and Trustworthiness in Retrieval-Augmented Generation. *arXiv preprint arXiv:2501.00879* (2025).
 - [35] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv preprint arXiv:2307.15043* (2023).
 - [36] W. Zou, R. Geng, B. Wang, and J. Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *34th USENIX Security Symposium (USENIX Security 25)*. 3827–3844.

Ethical Considerations and Generative AI Usage

This work is defensive in orientation. All experiments use benign proxy targets (e.g., forcing an “answer is unknown” response) rather than harmful content, rely on publicly available datasets and open-weight models, and involve no human subjects or personal data. The attacks studied are drawn from prior published work and are used only to evaluate defenses; no new attack capability is released. Generative AI tools were used solely for editorial assistance (language and formatting), and all outputs were verified by the authors for accuracy and originality.